

TEMA 6. Genericidad

1. Empleando `void*` en C o `Object` en Java, pon un ejemplo de una estructura de datos, que empleando un array primitivo, permita alojar cualquier tipo de dato.

EJEMPLO DE VOID EN C

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    void** data;
    int size;
    int capacity;
} ArrayGenerico;

ArrayGenerico* crear(int capacity) {
    ArrayGenerico* arr = malloc(sizeof(ArrayGenerico));
    arr->data = malloc(sizeof(void*) * capacity);
    arr->size = 0;
    arr->capacity = capacity;
    return arr;
}

void add(ArrayGenerico* arr, void* elemento) {
    if (arr->size < arr->capacity) {
        arr->data[arr->size++] = elemento;
    }
}

void* get(ArrayGenerico* arr, int index) {
    return arr->data[index];
}
```

USO->

```

int main() {
    ArrayGenerico* arr = crear(10);

    int a = 5;
    float b = 3.14;
    char* texto = "hola";

    add(arr, &a);
    add(arr, &b);
    add(arr, texto);

    printf("%d\n", *(int*)get(arr, 0));
    printf("%f\n", *(float*)get(arr, 1));
    printf("%s\n", (char*)get(arr, 2));

    return 0;
}

```

EJEMPLO OBJECT EN JAVA

```

public class ArrayGenerico {
    private Object[] data;
    private int size;

    public ArrayGenerico(int capacity) {
        data = new Object[capacity];
        size = 0;
    }

    public void add(Object elemento) {
        data[size++] = elemento;
    }

    public Object get(int index) {
        return data[index];
    }
}

```

USO->

```

public class Main {
    public static void main(String[] args) {
        ArrayGenerico arr = new ArrayGenerico(10);

        arr.add(5);           // autoboxing → Integer
        arr.add(3.14);        // Double
        arr.add("hola");       // String

        int a = (int) arr.get(0);
        double b = (double) arr.get(1);
        String s = (String) arr.get(2);

        System.out.println(a);
        System.out.println(b);
        System.out.println(s);
    }
}

```

2. Brevemente, ¿Qué significa la **programación genérica**? ¿Es el ejemplo anterior un ejemplo básico de programación genérica?

La programación genérica es un paradigma que permite escribir código independiente del tipo de dato, usando parámetros de tipo (como en Java). Así puedes crear estructuras y algoritmos reutilizables con seguridad de tipos en compilación, sin necesidad de hacer castings manuales.

En lugar de -> Object dato;

Se usa : T dato;

Donde T es un tipo que se especifica al usar la clase : ArrayGenerico numeros;

ArrayGenerico textos;

¿El ejemplo anterior es programación genérica?

No .

El ejemplo con Object en Java (o void* en C):

-> Permite almacenar cualquier tipo

-> No es programación genérica real

-> No tiene seguridad de tipos

-> Requiere castings manuales

Es más bien un enfoque “genérico a lo bruto”, pero no usa el mecanismo formal de genéricos.

3. Indica los problemas respecto al chequeo de tipos, de emplear `void*` o `Object` cuando se crean estructuras de datos genéricas.

CARACTERISTICAS

Detención de errores:

-> Uso de `void/Object`: En tiempo de ejecución (tarde y peligroso).

-> Genéricos: En tiempo de compilación (temprano y seguro).

Conversión de tipos:

-> Uso de `void/Object`: Requiere Casting explícito manual.

-> Genéricos: El compilador lo gestiona de forma implícita y segura.

Homogeneidad:

-> Uso de `void/Object`: Difícil de garantizar (permite mezclar tipos).

-> Genéricos: Estricta (garantiza que todos los elementos sean del mismo tipo).

4. Vamos entonces con mecanismos de mejora de la programación genérica ¿Qué son los **parámetros de tipo?**

Los parámetros de tipo son el corazón de la programación genérica: permiten escribir código que funciona con distintos tipos de datos sin tener que duplicarlo.

En pocas palabras, un parámetro de tipo es como una “variable de tipo”. En lugar de decir “esto es un `int` o un `String`”, dices “esto es de tipo `T` (o cualquier nombre)”, y ese tipo se concretará más adelante cuando se use el código.

```
public int suma(int a, int b) {  
    return a + b;  
}
```

```
public <T extends Number> double suma(T a, T b) {  
    // Convertimos ambos números a double para poder sumarlos  
    return a.doubleValue() + b.doubleValue();  
}
```

5. En Java existe "generics", en C++ existen "templates". Pon un ejemplo de uso de programación genérica en ambos, instanciando una lista o vector dinámico que solo admite `String`. Introduce valores, y luego haz un recorrido de ellos mostrando cómo cada elemento es del tipo concreto con seguridad.

EJEMPLO EN JAVA:

```

import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        // 1. Instanciamos la lista dinámica indicando que SOLO admite String
        ArrayList<String> listaSabores = new ArrayList<>();

        // 2. Introducimos valores (el compilador verifica que sean String)
        listaSabores.add("Vainilla");
        listaSabores.add("Chocolate");
        listaSabores.add("Fresa");

        // Si intentaras hacer: listaSabores.add(125); -> El compilador daría ERROR.

        // 3. Recorrido seguro: Cada elemento se extrae directamente como String
        // No hace falta poner (String) listaSabores.get(i)
        for (String sabor : listaSabores) {
            System.out.println("Sabor: " + sabor);
        }
    }
}

```

EJEMPLO EN C++

->Usando Templates

```

#include <iostream>
#include <vector>
#include <string>

int main() {
    // 1. Instanciamos el vector dinámico indicando que SOLO admite string
    // Ahí estás invocando la plantilla (template) que los creadores de C++ ya programaron dentro
    std::vector<std::string> listaSabores;

    // 2. Introducimos valores
    listaSabores.push_back("Vainilla");
    listaSabores.push_back("Chocolate");
    listaSabores.push_back("Fresa");

    // Si intentaras hacer: listaSabores.push_back(125); -> El compilador daría ERROR.

    // 3. Recorrido seguro: El bucle extrae cada elemento como un string concreto
    for (const std::string& sabor : listaSabores) {
        std::cout << "Sabor: " << sabor << std::endl;
    }

    return 0;
}

```

6. Sobre el funcionamiento de la programación genérica. ¿Qué hace el compilador cuando se instancia una clase que tiene parámetros de tipo? ¿Hace lo mismo C++ y Java? ¿Qué es el "type erasure" de Java y la "instanciación de plantillas" de C++?

No exactamente: Java y C++ no hacen lo mismo cuando usas genéricos. De hecho, siguen dos estrategias casi opuestas.

- ◆ ¿Qué hace el compilador al instanciar genéricos?

Depende del lenguaje:

JAVA:

Java prioriza la compatibilidad hacia atrás y el ahorro de espacio. Su estrategia consiste en usar los

genéricos solo como un escudo de protección mientras escribes el código, pero los destruye antes de que el programa se ejecute.

()->¿Que hace el compilador?

Cuando compilas tu código, el compilador de Java realiza el proceso de Borrado de Tipos (Type Erasure):

Elimina todos los parámetros de tipo (las).

Sustituye cada T por la clase base Object (o por el límite que le hayas puesto, si usaste extends).

Inserta castings automáticos y ocultos cada vez que sacas un elemento de la estructura para garantizar que no tengas que escribirlos tú.

```
// Código que tú escribes
class Caja<T> {
    private T contenido;
    public T get() { return contenido; }
}

// Código real que se ejecuta (Bytecode)->El compilador lo transforma internamente en esto antes:
class Caja {
    private Object contenido; // T se convirtió en Object
    public Object get() { return contenido; }
}
```

Si luego en tu programa haces `String texto = caja.get();`, el compilador introduce un cast invisible por debajo: `String texto = (String) caja.get();`

C++ ("Instanciación de Plantillas" (Mapeo de Código))

C++ prioriza el rendimiento máximo y la velocidad de ejecución. Su estrategia consiste en usar la plantilla como un molde de repostería y clonar la estructura de datos de forma real para cada tipo que necesites.

()->¿Qué hace el compilador?

Cuando usas un template, el compilador no genera código inmediatamente. Espera a ver cómo lo usas en el main. El proceso de Instanciación de plantillas funciona así:

El compilador detecta qué tipos de datos específicos estás solicitando (por ejemplo, int y double).

Duplica el código de la clase en silencio tantas veces como tipos diferentes hayas usado.

Sustituye la T de forma física y permanente por el tipo real en cada copia.

Paso 1: Si tu defines esta plantilla:


```
template <typename T>
class Caja {
    T contenido;
};
```

En este punto, la clase Caja no existe realmente en el programa; es solo un concepto.

Paso 2: El compilador analiza tu main (Las Órdenes)

```
int main() {
    Caja<int> cajaEnteros;    // <-- El compilador detecta que pides una Caja de enteros
    Caja<double> cajaDecimales; // <-- El compilador detecta que pides una Caja de decimales
    return 0;
}
```

Paso 3: La Instanciación (La duplicación en silencio)

```
// Copia 1: Generada automáticamente para 'int'
class Caja_int {
    int contenido; // La 'T' ha sido sustituida por 'int'
};
```

Paso 4: Segunda Instanciación (Nueva duplicación)

```
// Copia 2: Generada automáticamente para 'double'
class Caja_double {
    double contenido; // La 'T' ha sido sustituida por 'double'
};
```

Resultado Final (Lo que realmente se compila)

Una vez que el compilador ha terminado de escanear todo tu código, "borra" tu plantilla original y la reemplaza por las clases específicas que ha fabricado.

El código real y definitivo que se convierte en el programa ejecutable es el siguiente

```
// --- CÓDIGO FINAL REAL GENERADO POR EL COMPILADOR ---
```

```
class Caja_int {  
    int contenido;  
};  
  
class Caja_double {  
    double contenido;  
};  
  
int main() {  
    Caja_int cajaEnteros;    // Ahora usa la clase de enteros  
    Caja_double cajaDecimales; // Ahora usa la clase de decimales  
    return 0;  
}
```

◆ Diferencia clave (la idea importante)

Java → genéricos “falsos” en runtime (borrados)

C++ → genéricos “reales” (especialización por tipo)

7. Vamos a crear una nueva clase con parámetros de tipo. Define en Java una clase `Par`, que permite alojar dos valores de tipos diferentes. Incluye un constructor y un getter para cada tipo. Pon un ejemplo de uso de ese `Par`, por ejemplo para especificar el tipo de retorno de una función que

devuelve en un Par la media y desviación típica de un array de double .

```
class Par<Q, P>{
    private final P primero;
    private final Q segundo;

    public Par (P primero, Q segundo){
        this.primerο= primero;
        this.segundo= segundo;
    }
    public Q getSegundo(){
        return this.segundo;
    }
    public P getPrimero(){
        return this.primerο;
    }
}

class Estadisticas{
    public static Par<Double, Double> mediaYDesviacionTipica(List<Double> valores){
        //...
        double media=....
        double stddev=...

        return new Par<>(media, stddev);
    }

    //uso media y desviacion tipica mediaYDesviacionTipica
    List<Double>valores=....;
    Par <Double, Double >mediaYDesviacionTipica= mediaYDesviacionTipica(valores);
    double media = mediaYDesviacionTipica.getPrimero();
    Par resultadoAlumno=...
    Alumno alumno= resultadoAlumno.getPrimero();
}
////////////////////////////////////
class Par<P,Q>{
    private Object primero;
    private Object segundo;

    public Par(Object primero, Object segundo){
```

```

        this.primeros= primeros;
        this.segundos= segundos;
    }

    public P getPrimeros

}

////////////////////////////////////
class Colegio{
    public static List<Par<Alumno, Double>>obtenerAlumnosYsusNotas(){

    }

}

```

8. En Java, se pueden declarar parámetros de tipo también a nivel de método, no solo a nivel de clase. Pon un ejemplo con un método genérico seleccionaUno , que pasados dos objetos del mismo tipo, te devuelva aleatoriamente uno de ellos. Muestra la diferencia de definirlo con dos Object , a definirlo con dos parámetros de tipo, en terminos de (i) evitar downcasting y (ii) forzar que ambos objetos sean del mismo tipo.

- ◆ Version SIN genericos (usando Object)

```

import java.util.Random;

public class EjemploSinGenericos {

    public static Object seleccionaUno(Object a, Object b) {
        Random rand = new Random();
        return rand.nextBoolean() ? a : b;
    }

    public static void main(String[] args) {
        String s = "Hola";
        Integer n = 42;

        // Permite mezclar tipos distintos
        Object resultado = seleccionaUno(s, n);

        // Necesita downcasting
        String texto = (String) resultado; // Puede fallar en tiempo de ejecución
        System.out.println(texto);
    }
}

```

Problemas:

Necesita downcasting

No garantiza que ambos parámetros sean del mismo tipo

Puede lanzar ClassCastException

- ◆ Versión CON método genérico

```

import java.util.Random;

public class EjemploConGenericos {

    public static <T> T seleccionaUno(T a, T b) {
        Random rand = new Random();
        return rand.nextBoolean() ? a : b;
    }

    public static void main(String[] args) {
        String s1 = "Hola";
        String s2 = "Adiós";

        // Solo acepta argumentos del mismo tipo
        String resultado = seleccionaUno(s1, s2);

        // No necesita casting
        System.out.println(resultado);

        // Esto NO compila (tipos distintos)
        // seleccionaUno("Hola", 42);
    }
}

```

◆ Diferencias clave

(i) Evitar downcasting

Con Object → necesitas cast:

String texto = (String) resultado;

Con genéricos → no hace falta:

String resultado = seleccionaUno(s1, s2);

(ii) Forzar mismo tipo

Con Object → permite mezclar tipos:

seleccionaUno("Hola", 42); // ✓ compila (pero es peligroso)

Con → obliga a que sean del mismo tipo:

seleccionaUno("Hola", 42); // error de compilación

9. ¿Se pueden establecer restricciones en los parámetros de tipo? Por ejemplo, si quiero definir un

tipo genérico `<T>` , ¿puedo decir que tenga que ser, al menos, un número para poder tratarlo como tal? Pon un ejemplo en Java de un `Punto` con dos coordenadas, metodos `getX` , `getY` , y una función `calcularDistanciaA` otro `Punto` . Permite que esas coordenadas sean cualquier tipo de número. Pon dos soluciones: una simplemente creando coordenadas de tipo `Number` y otra añadiendo generics para reforzar el chequeo de tipos y saber exactamente con qué tipo de número trabaja el `Punto` . En este caso y respecto al "type erasure", ¿cuál es el tipo final tras la compilación?

- ◆ Solución 1: usando directamente `Number`

```

public class PuntoNumber {

    private Number x;
    private Number y;

    public PuntoNumber(Number x, Number y) {
        this.x = x;
        this.y = y;
    }

    public Number getX() {
        return x;
    }

    public Number getY() {
        return y;
    }

    public double calcularDistanciaA(PuntoNumber otro) {
        double dx = this.x.doubleValue() - otro.x.doubleValue();
        double dy = this.y.doubleValue() - otro.y.doubleValue();
        return Math.sqrt(dx * dx + dy * dy);
    }

    public static void main(String[] args) {
        PuntoNumber p1 = new PuntoNumber(1, 2);           // Integer
        PuntoNumber p2 = new PuntoNumber(3.5, 4.5);        // Double

        double d = p1.calcularDistanciaA(p2);
        System.out.println(d);
    }
}

```

- ◆ Solución 2: usando genéricos con restricción


```

public class Punto<T extends Number> {

    private T x;
    private T y;

    public Punto(T x, T y) {
        this.x = x;
        this.y = y;
    }

    public T getX() {
        return x;
    }

    public T getY() {
        return y;
    }

    public double calcularDistanciaA(Punto<T> otro) {
        double dx = this.x.doubleValue() - otro.x.doubleValue();
        double dy = this.y.doubleValue() - otro.y.doubleValue();
        return Math.sqrt(dx * dx + dy * dy);
    }

    public static void main(String[] args) {
        Punto<Integer> p1 = new Punto<>(1, 2);
        Punto<Integer> p2 = new Punto<>(3, 4);

        double d = p1.calcularDistanciaA(p2);
        System.out.println(d);

        // ✗ Esto NO compila (tipos distintos)
        // Punto<Integer> p3 = new Punto<>(1, 2);
        // Punto<Double> p4 = new Punto<>(3.0, 4.0);
        // p3.calcularDistanciaA(p4);
    }
}

```

◆ Diferencias clave

✓ Con Number

Permite mezclar tipos (Integer, Double, etc.)

Menos seguro

Más flexible pero menos preciso

✓ Con

Obliga a trabajar con un único tipo concreto

Mayor seguridad en compilación

Mejora la expresividad del código

10. Sobre las soluciones anteriores. Si bien ambas permiten trabajar con distintos tipos de número sin duplicar la clase `Punto`, reflexiona sobre el refuerzo del chequeo de tipos con generics. ¿Permiten ambas crear un punto con una coordenada de tipo entero y la otra coordenada de tipo real? ¿Qué tipo devuelve el `getX` con la solución sin generics y qué tipo devuelve el que tiene la solución con generics?

(1)-> ¿Permiten crear un punto mezclando coordenadas (X entera e Y decimal)?

● Solución SIN Genéricos (Number)

Sí lo permite. Dado que las variables internas son simplemente de tipo `Number`, el compilador te deja meter cualquier subclase de número en cualquier posición.

```
// Código válido pero "peligroso"
PuntoNumber p = new PuntoNumber(5, 4.5); // X es Integer, Y es Double
```

Problema: No hay control de homogeneidad. Pierdes la simetría matemática de que ambas coordenadas pertenezcan al mismo conjunto numérico.

● Solución CON Genéricos ()

NO lo permite. Al declarar la clase como `Punto`, obligas a que tanto la coordenada X como la Y utilicen exactamente el mismo tipo concreto T.

```
Punto<Integer> p1 = new Punto<>(1, 2);    // Válido (ambos son Integer)
Punto<Double> p2 = new Punto<>(1.0, 2.5); // Válido (ambos son Double)
```

```
// ✗ ERROR DE COMPILACIÓN: No puedes mezclar tipos para la misma 'T'
Punto<Integer> p3 = new Punto<>(1, 2.5); // El compilador frena el programa aquí
```

Ventaja: El compilador blinda la estructura garantizando que el punto sea matemáticamente coherente.

(2)-> ¿Qué tipo de dato devuelve el método getX()?

● Solución SIN Genéricos (Number)

El método está firmado como `public Number getX()`. Por lo tanto, devuelve un objeto genérico de tipo `Number`.

Consecuencia: Al recuperar la coordenada, has perdido la información de qué era originalmente. Si necesitas operar con ella como un entero, te ves obligado a arriesgarte con un downcasting manual:

```
PuntoNumber p = new PuntoNumber(5, 10);

// El compilador te obliga a hacer un cast manual (peligroso)
Integer x = (Integer) p.getX();
```

● Solución CON Genéricos ()

El método está firmado como `public T getX()`. Por lo tanto, devuelve el tipo exacto con el que instanciaste la clase.

Contraste: No se pierde la información del tipo en el código. El compilador recuerda perfectamente qué tipo de punto creaste y te devuelve ese tipo directamente, eliminando los castings:

```
Punto<Integer> p = new Punto<>(5, 10);

// 100% Seguro: getX() ya devuelve un Integer de forma nativa
Integer x = p.getX();
```

11. Hagamos un ejemplo avanzado. El siguiente código, con interfaz `Punto`, que define un método `calcularDistanciaA(Punto p)`, junto con las implementaciones `Punto2D` y `Punto3D`. Añade generics para asegurarnos que la sobreescritura del método `calcular distancia a otro Punto` siempre es sobre un

Punto del mismo tipo, evitando instanceof y el downcasting.

```
public interface Punto {
    public double distanciaA(Punto p);
}

public class Punto2D implements Punto {
    private final double x, y;
    public Punto2D(double x, double y) {
        this.x = x; this.y = y;
    }

    @Override
    public double distanciaA(Punto p) {
        if (p instanceof Punto2D) {
            Punto2D p2d = (Punto2D) p;
            return Math.sqrt(Math.pow(x - p2d.x, 2)
                               + Math.pow(y - p2d.y, 2));
        } else {
            throw new RuntimeException("p debe ser Punto 2D");
        }
    }
}

public class Punto3D implements Punto {
    // Igual que Punto2D, pero con tres coordenadas
    ...
}
```

12. Dado que String es subtipo de Object, ¿significa eso que List<String> es subtipo de List<Object>? ¿Y que String[] es subtipo de Object[]? Razona por qué la respuesta es diferente en cada caso y qué problema en tiempo de ejecución puede aparecer con los arrays. A partir de estos ejemplos, define qué significa que un tipo genérico sea **covariante,**

contravariante o invariante respecto a su parámetro de tipo.

(1)-> ¿significa eso que `List<String>` es subtipo de `List<Object>` ?

Aunque sabemos que un `String` es un subtipo de `Object`, la relación no se hereda de la misma forma en las colecciones y en los arrays.

Caso A: `List` y `List`